

Multiple Choice - choose the best answer

1. (1 point) Which of the following code segments have run-time that is $O(N \log(N))$, but not $O(N)$?

a)

```
int sum = 0;
for (int i = 0; i < N; i++)
    for (int j = i; j > 0; j /= 2)
        sum++;
```

b)

```
int sum = 0;
for (int i = 1; i < N; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

c)

```
int sum = 0;
for (int i = N; i > 0; i /= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

d)

```
int sum = 0;
for (int i = 1; i < N; i *= 2)
    for (int j = 0; j < N; j++)
        sum++;
```

- e) All of the above.
f) None of the above.

2. (1 point) What is the most accurate worst-case run-time of the **find** operation in the **Union-Find** data structure using the **QuickFind** algorithm?

- a) $O(n^2)$
b) $O(\log(n))$
c) $O(n)$
d) $O(1)$
e) None of the above.

3. (1 point) What is the most accurate worst-case run-time of **Mergesort** for an array with n items?

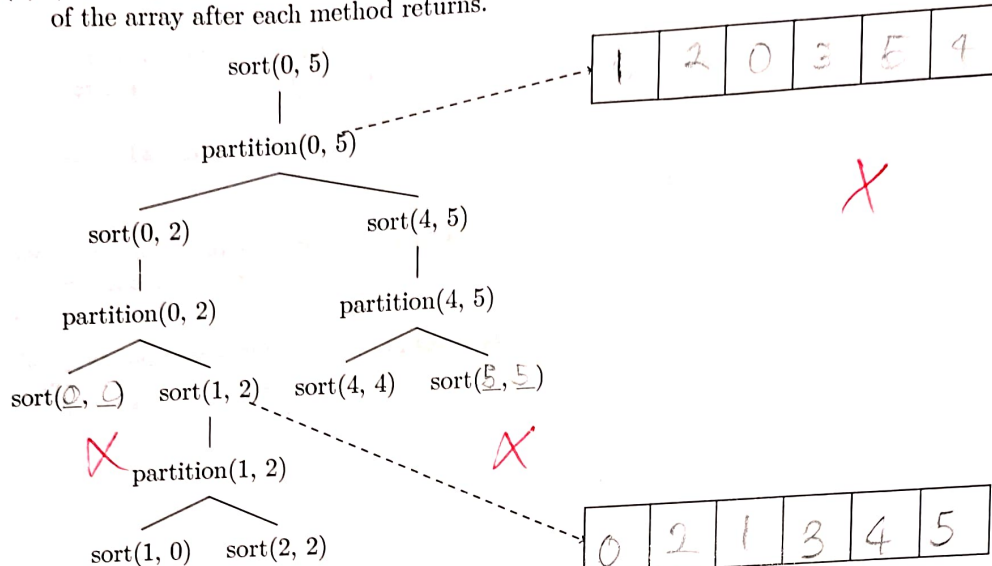
- a) $O(\log(n))$
b) $O(n)$
c) $O(n \log(n))$
d) $O(n^2)$
e) None of the above.

4. (1 point) Which of the following inputs results in the worst-case run-time for **Quicksort** algorithm when partitioning is based on the first element (i.e., no shuffle)?

- a) [100, 99, 98, 97, 96].
b) [99, 100, 96, 97, 98].
c) [100, 100, 100, 100, 100].
d) All of the above.
e) None of the above.

5. Given the array [3, 1, 2, 0, 5, 4], if we apply the **QuickSort** algorithm using the first element as the pivot, the execution will follow the tree structure shown below. However, some arguments are missing in the method invocation.

- (a) (2 points) Fill in these missing arguments appropriately.
 (b) (4 points) Update the two diagrams corresponding to two method calls to reflect the state of the array after each method returns.



6. (4 points) Assume the **MaxPQ** class implements a maximum-oriented priority queue by storing items in an **array** called **data**. Fill in the below diagram of **data** after all of the below code has been executed (i.e., after line 11).

```

1 MaxPQ<Integer> myPQ;
2 myPQ.insert(0);
3 myPQ.insert(13);
4 myPQ.insert(13);
5 myPQ.removeMax();
6 myPQ.removeMax();
7 myPQ.insert(20);
8 myPQ.insert(17);
9 myPQ.insert(14);
10 myPQ.removeMax();
11 myPQ.insert(18);

```

0	17	14	18
---	----	----	----

data[0] data[1] data[2] data[3]

Circle your answer for the following questions based on the organization of your array:

1. What type of array did you use here?

- a) Sorted
 (b) Unsorted

2. What is run-time of **insert**?

- (a) $O(1)$
 b) $O(\log(n))$
 c) $O(n)$
 d) $O(n \log(n))$
 e) $O(n^2)$

3. What is run-time of **removeMax**?

- a) $O(1)$
 b) $O(\log(n))$
 (c) $O(n)$
 d) $O(n \log(n))$
 e) $O(n^2)$

7. Below is a buggy implementation of the sort method for Mergesort with an optimization.

```

1 void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
2     if (hi <= lo) return;
3     int mid = lo + (hi - lo) / 2;
4     if (less(a[mid], a[mid+1])) return;
5
6     sort(a, aux, lo, mid);
7     sort(a, aux, mid + 1, hi);
8
9     merge(a, aux, lo, mid, hi);
10 }
11
12 boolean less(Comparable v, Comparable w) {
13     return v.compareTo(w) < 0;
14 }

```

(a) (1 point) Identify the bug(s) in this code.

The bug is in line 4

(b) (2 points) Fix the code.

Insert line 4 in between line 7 and 9 so it looks like

```

sort(a, aux, lo, mid);
sort(a, aux, mid+1, hi);
if (less(a[mid], a[mid+1])) return;
merge(a, aux, lo, mid, hi);

```

(c) (1 point) How does the fixed code optimize Mergesort? (i.e., when does it speed up the computation?) It is used to check if the array is already sorted

8. (7 points) You are tasked with designing a custom data type to manage a simple music queue (note: this is different from a standard queue). The `MusicQueue` class should support the following functionalities:

- `playNow(song)`: Immediately starts playing the given song **without modifying the queue**. (1 mark)
- `playNext(song)`: Adds a song to the front of the queue, making it the next song to be played. (2 marks)
- `playLast(song)`: Adds a song to the end of the queue. (2 marks)
- `next()`: Moves to the next song in the queue, making it the currently playing song. (2 marks)

Your task is to fill in the space provided to do the following:

1. Implement the `MusicQueue` class using a **singly linked list**.
2. Ensure that `playNext()`, `playLast()`, and `next()` correctly modify the queue structure.
3. Ensure all methods must run in constant time (i.e., $O(1)$).

Keep in mind:

- `MusicQueue` has no constructor, so all object instance fields will be initialized to `null`.
- Most of the methods should have behave differently when the `MusicQueue` contains no songs compared to when it contains at least one song.
- If something unexpected happens (like `next()` is called when the `MusicQueue` is empty), you do not have to throw an exception, but your code should handle these cases without crashing.
- You may assume that a Java class `Song` exists and is accessible to your code, but you shouldn't need anything from that class other than the `Song` type it provides.
- You can assume the following private Java class is contained in `MusicQueue`:

```

1 private class Node{
2     Song data;
3     Node next;
4     public Node(Song s){
5         data = s;
6         next = null;
7     }
8     public Node(Song s, Node n){
9         data = s;
10        next = n;
11    }
12 }

```

For example if a `MusicQueue m` is initially in the state:

Now Playing:

Song1

Up Next:

Song2

Song3

Song4

and the following are done in order : `m.next()`, `m.playNext(Song5)`, `m.playLast(Song6)`, `m.playNow(Song7)`, `m.next()`, `m` will change as follows:

1. Now Playing:	2. Now Playing:	3. Now Playing:	4. Now Playing:	5. Now Playing:
Song2	Song2	Song2	Song7	Song5
Up Next:	Up Next:	Up Next:	Up Next:	Up Next:
Song3	Song5	Song5	Song5	Song3
Song4	Song3	Song3	Song3	Song4
	Song4	Song4	Song4	Song6
		Song6	Song6	

In the space provided over the next two pages, write your implementation of `MusicQueue` in Java.

```
1 public class MusicQueue {  
2     private Song nowPlaying;  
3     private Node upNext, upLast;  
4  
5
```

```
6     public void playNow(Song s){  
7         now_playing = s;  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19     }
```

```
21  
22     public void next(){  
23         while (up_next != null){  
24             now_playing = up_next;  
25             }  
26             up_next = up_next.next;  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37     }
```

```
38  
39  
40     public void playNext(Song s){  
41         up_next.next = up_next;  
42         up_next = s;  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55
```

```
64 public void playLast(Song s){  
65     private Node container;  
66     while (up_next.next != null){  
67         container = up_next;  
68     }  
69     container.next = s;  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88 }
```